ΠΑΝΕΠΙΣΤΗΜΙΟ ΔΥΤΙΚΗΣ ΜΑΚΕΔΟΝΙΑΣ

8 BIT CPU

ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΥΠΟΛΟΓΙΣΤΩΝ

ΟΝΟΜΑ: ΓΕΩΡΓΙΟΣ ΠΑΠΑΜΙΧΑΗΛ

Α.Μ.:2221

ΕΞΑΜΗΝΟ: 5ο

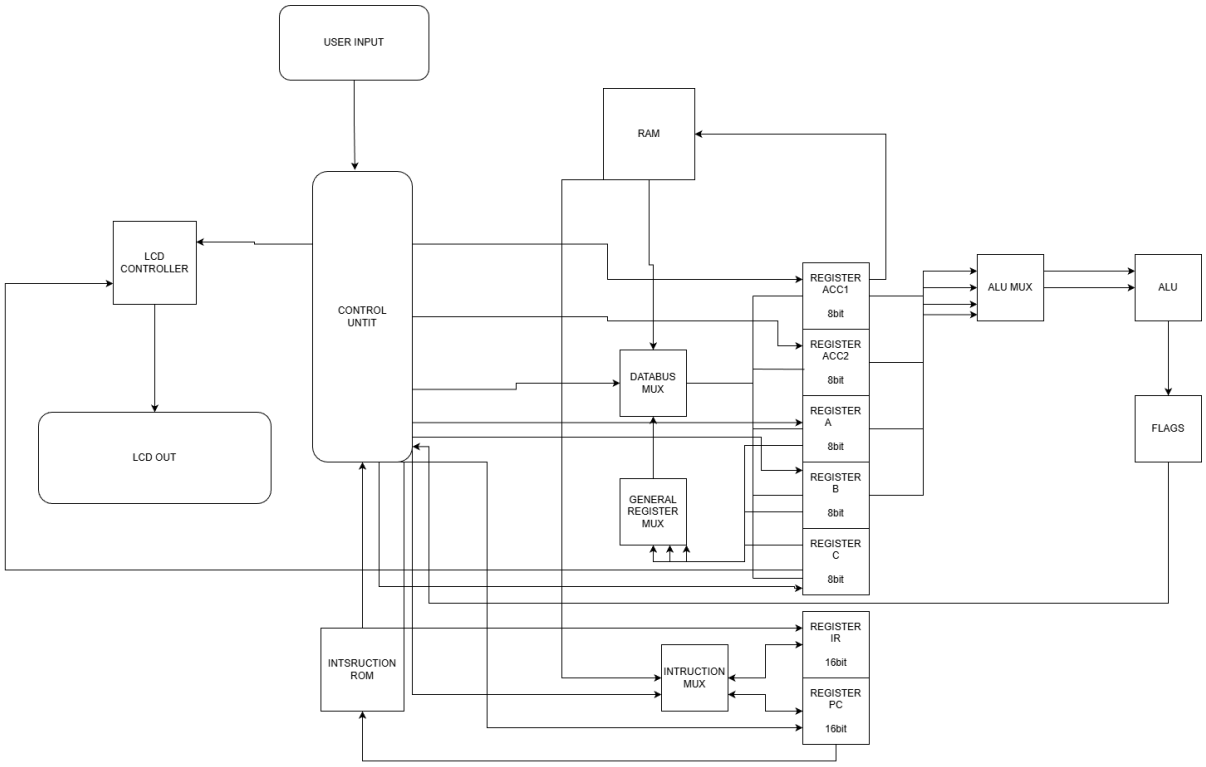
1. Εισαγωγή.....1
2. Αρχιτεκτονική.....1
3. Εντολές.....5
4. Καταχωρητές.....
5. ALU.....
6. Διαδρομή Δεδομένων.....
7. Control Unit.....
8. Assembler.....
9. Παραδείγματα κώδικα.....

1 Εισαγωγή

Το παρόν έγγραφο παρουσιάζει την αρχιτεκτονική του δικού μου προσαρμοσμένου 8-bit επεξεργαστή και παρέχει βασικές πληροφορίες σχετικά με το πλαίσιο και τη λειτουργία του. Ο επεξεργαστής χρησιμοποιεί 256 διευθύνσεις RAM, καθεμία από τις οποίες αποθηκεύει ένα byte, από τη διεύθυνση 0x00 έως τη διεύθυνση 0xFF.

Σκοπός του εγγράφου είναι να αναδείξει τις θεμελιώδεις μονάδες του επεξεργαστή, ξεκινώντας από τα υποσυστήματα του και καταλήγοντας στη συνολική αρχιτεκτονική.

Αρχιτεκτονική



Η αρχιτεκτονική του επεξεργαστή αποτελείται από τις βασικές μονάδες που συνεργάζονται για την εκτέλεση των εντολών. Το διάγραμμα παρουσιάζει τη συνολική ροή δεδομένων και ελέγχου μέσα στην CPU.

Καταχωρητές:

- Γενικής χρήσης: 2 καταχωρητές A και B, 8-bit, για προσωρινή αποθήκευση δεδομένων.

- Ειδικής χρήσης: 3 καταχωρητές ACC1, ACC2, C, 8-bit, που χρησιμοποιούνται από την ALU για υπολογισμούς και το LCD controll για εκτύπωση.
- Εσωτερικής χρήσης: 2 καταχωρητές IR (Instruction Register) και PC (Program Counter), 16-bit, για αποθήκευση τρέχουσας εντολής και διεύθυνσης εντολής.
- Flags: Καταχωρητής carry και zero flag για αποθήκευση αποτελεσμάτων και ελέγχου κατάστασης.
- 50Mhz clock
-

ALU (Arithmetic Logic Unit):
Υποστηρίζει 8 βασικές πράξεις:

- Πρόσθεση, Αφαίρεση, OR, AND, XOR, NOT, Left Shift, Right Shift.

Μνήμη:

- ROM: Αποθηκεύει τις εντολές του προγράμματος.
- RAM: 256 διευθύνσεις των 8-bit (1 byte ανά διεύθυνση), για προσωρινή αποθήκευση δεδομένων.

Μονάδες Ελέγχου και Εισόδου/Εξόδου:

- Μονάδα Ελέγχου Εντολών: Παράγει τα σήματα ελέγχου για την εκτέλεση των εντολών.
- Μονάδα Εγγραφής σε LCD Οθόνη: Επιτρέπει την εμφάνιση αποτελεσμάτων.
- 8-bit Είσοδος Χρήστη: Δέχεται δεδομένα από τον χρήστη.

Λειτουργία γενικά:
Η εκτέλεση κάθε εντολής γίνεται σε τρία βασικά βήματα:

1. Ο PC φορτώνεται στον IR και μετά αφξάνεται κατα 1.
2. Η Μονάδα Ελέγχου αποκωδικοποιεί την εντολή και ενεργοποιεί τα σωστά σήματα για την ALU και τους καταχωρητές.
3. Η ALU εκτελεί την πράξη και ενημερώνει τους καταχωρητές και τα flags.

3 Εντολές

Ο επεξεργαστής υποστηρίζει διάφορες εντολές, οι οποίες χωρίζονται σε κατηγορίες ανάλογα με τη λειτουργία τους: μεταφορά δεδομένων, αριθμητικές/λογικές πράξεις, shift, σύγκριση, άλματα, είσοδο/έξοδο και έλεγχος προγράμματος.

1. Μεταφορά Δεδομένων

MOV Register, VALUE_IN_CODE. Μεταφορά τιμής από τον κώδικα στο καταχωρητή. Register: ACC1, ACC2, A, B, C

LD Register, *<διεύθηνση RAM>. Μεταφορά της τιμής διεύθηνσης RAM στον καταχωρητή.

Register: ACC1, ACC2, A, B, C

ST *<διεύθηνση RAM>, Registar. Μεταφορά καταχωρητή στην θέση της διεύθηνσης RAM.

Register: ACC1, A, B, C

MOVR Register1, Register2. Εγγραφή του Register2 στον Register1. Register: A, B, C

Στον assembler ολες οι εντωλες μετακήνησης ονομάζονται MOV.

2. Αριθμητικές / Λογικές Πράξεις

ADD Register. Αποθήκευση της πρόσθεση του ACC1 + ACC2 στον Register καταχωρητή

SUB Register. Αποθήκευση της πρόσθεση του ACC1 - ACC2 στον Register καταχωρητή

OR Register. Αποθήκευση της πρόσθεση του ACC1 | ACC2 στον Register καταχωρητή

AND Register. Αποθήκευση της πρόσθεση του ACC1 & ACC2 στον Register καταχωρητή

NOT Register. Αποθήκευση της πρόσθεση του ~ACC1 στον Register καταχωρητή

XOR Register. Αποθήκευση της πρόσθεση του ACC1 xor ACC2 στον Register καταχωρητή

INC Register. Αύξηση κατά 1 του ACC1 καταχωρητή

DINC Register. Μείωση κατά 1 του ACC1 καταχωρητή

ADD Register1, Register2. Αποθήκευση της πρόσθεση του ACC1 + ACC2 στον Register καταχωρητή

SUB Register1, Register2. Αποθήκευση της πρόσθεση του Register1 - Register2 στον Register1 καταχωρητή

OR Register1, Register2. Αποθήκευση της πρόσθεση του Register1| Register2 στον Register1 καταχωρητή

AND Register1, Register2. Αποθήκευση της πρόσθεση του Register1& Register2 στον Register1 καταχωρητή

NOT Register1, Register1. Αποθήκευση της πρόσθεση του ~ Register1 στον Register1 καταχωρητή

XOR Register1, Register2. Αποθήκευση της πρόσθεση του Register1 xor Register2 στον Register1 καταχωρητή

INC Register1, Register1. Αύξηση κατά 1 του Register καταχωρητή

DINC Register1, Register1. Μείωση κατά 1 του Register καταχωρητή

SHL Register, N. Shift αριστερά N φορες του Register καταχωρητή.

SHR Register, N. Shift δεξιά N φορες του Register καταχωρητή.

4. Σύγκριση

CMP _1, _2. Τα _1, _2 μπορούν να ειναι οι καταχωριτές, τιμές στην εντολή(ROM) ή τιμές απο την RAM. Το _1 μετακινήται στο ACC1 και το _2 Στον ACC2 και γίνεται η πράξη ACC1-ACC2 όπου αναβαθμίζει τα flags.

5. Άλματα

<TAG>:. Το TAG ονομάζεται από τον χρήστη στον κώδικα και αναγνωρίζεται από τον assembler μέσω των :. Ο assembler συνδέει την διεύθηνση της επόμενης εντολής με το TAG.

JMP <TAG>. Άμεσο άλμα στην διεύθυνση <TAG>.

JA <TAG>. Άλμα αν μεγαλύτερο.

JB <TAG>. Άλμα αν μικρότερο.

JE <TAG>. Άλμα αν ισότητα.

JEA <TAG> Άλμα αν μεγαλύτερο ή ίσο.

JEB <TAG> Άλμα αν μικρότερο ή ίσο.

JC <TAG> Άλμα αν το carry flag είναι 1.

JZ <TAG> Άλμα αν το zero flag είναι 1.

6. Είσοδος / Έξοδος

IN Register. Λήψη δεδομένου από τον χρήστη στον Register καταχωρητή.

PRT. Εμφάνιση δεδομένου στην LCD.

CLR. Εκκαθάριση οθόνης

HLT. Διακοπή εκτέλεσης.

4 Καταχωρητές

Ο επεξεργαστής διαθέτει διάφορους καταχωρητές, χωρισμένους σε γενικής, ειδικής και εσωτερικής χρήσης, καθώς και καταχωρητές flags.

Τύπος	Όνομα	Μέγεθος	Λειτουργία
Γενικής χρήσης	A, B	8-bit	Αποθήκευση προσωρινών δεδομένων για υπολογισμούς και την μεταφορά τους
Ειδικής χρήσης	ACC1	8-bit	Αποθήκευση προσωρινών δεδομένων για υπολογισμούς. Αποθήκευση της τιμής που θα μεταφεθεί στην RAM
Ειδικής χρήσης	ACC2	8-bit	Αποθήκευση προσωρινών δεδομένων για υπολογισμούς.
Ειδικής χρήσης	C	8-bit	Αποθήκευση προσωρινών

			δεδομένων για την μεταφορά τους. Εκτήπωση των δεδομένων στο LCD
Εσωτερικής χρήσης		16-bit	Αποθήκευση της τρέχουσας εντολής
Εσωτερικής χρήσης		16-bit	Περιέχει τη διεύθυνση της επόμενης εντολής
Flags		1-bit	Παρακολούθηση αποτελεσμάτων της ALU για carry και μηδενικό αποτέλεσμα

Καταχωρητές

Η ALU αποτελεί τη βασική αριθμητική και λογική μονάδα του επεξεργαστή και είναι υπεύθυνη για την εκτέλεση όλων των πράξεων που απαιτούν υπολογισμό. Λειτουργεί αποκλειστικά με 8-bit δεδομένα και χρησιμοποιεί τους καταχωρητές **A, B, ACC1, ACC2 και C** ως πηγές και προορισμούς τιμών.

Υποστηριζόμενες Πράξεις

Η ALU υλοποιεί τις ακόλουθες 8 βασικές λειτουργίες:

ADD, SUB, OR, AND, XOR, NOT, SHL, SHR

Μετά από κάθε πράξη, η ALU ενημερώνει τους δύο καταχωρητές flags:

Carry (C): Ενεργοποιείται όταν υπάρχει υπερχείλιση σε πρόσθεση ή δανεισμός σε αφαίρεση.

Zero (Z): Ενεργοποιείται όταν το αποτέλεσμα της πράξης είναι 0.

Ροή Λειτουργίας

1. Η μονάδα ελέγχου στέλνει το κατάλληλο opcode.
2. Η ALU λαμβάνει τις τιμές από τους αντίστοιχους καταχωρητές (A, B ή ACC1/ACC2) μέσω ενός πολυπλέκτη.
3. Εκτελεί την πράξη που αντιστοιχεί στο opcode.

4. Το αποτέλεσμα αποθηκεύεται στον κατάλληλο καταχωρητή.
5. Γίνεται ενημέρωση των flags.

Χαρακτηριστικά

- Όλες οι πράξεις εκτελούνται σε 2 clock cycle.
- Η ALU συνεργάζεται στενά με τη μονάδα ελέγχου, η οποία καθορίζει την πράξη μέσω των σημάτων ελέγχου.
- Τα αποτελέσματα μπορούν να χρησιμοποιηθούν άμεσα σε επόμενες εντολές.

6 Διαδρομή Δεδομένων (Datapath)

Όλοι οι καταχωρητές A, B, C, ACC1, ACC2 διαθέτουν κοινή είσοδο **In**, η οποία προέρχεται από έναν πολυπλέκτη 3 εισόδων. Οι πηγές του multiplexer είναι:

1. **Ένας εσωτερικός πολυπλέκτης** που παρέχει τιμές από:
    a. A
    b. B
    c. C
    d. ALU output
    (επιλέγεται όταν τα δεδομένα μεταφέρονται μεταξύ καταχωρητών ή από ALU)
2. **Την έξοδο της RAM**
    (επιλέγεται για εντολές φόρτωσης από μνήμη)
3. **Σήμα από το Control Unit**
    (επιτρέπει εγγραφή άμεσων τιμών ή ειδικές τιμές κατά την εκτέλεση εντολών)

Η **RAM** γράφει δεδομένα που βρίσκονται στον **ACC1**, ενώ η **διεύθυνση μνήμης** προέρχεται από το πεδίο διεύθυνσης της εντολής στη ROM, το οποίο δίνεται απευθείας από το Control Unit.

Οι καταχωρητές **PC** και **IR** λαμβάνουν την είσοδό τους από δικό τους multiplexer, ο οποίος επιλέγει μεταξύ:

- τρέχουσας τιμής του **IR**
- τρέχουσας τιμής του **PC**
- άμεσης τιμής/διεύθυνσης από το **Control Unit** (χρησιμοποιείται για άλματα – jumps)

Ο καταχωρητής **C** (που κρατά μια 8-bit τιμή) συνδέεται απευθείας με τη μονάδα ελέγχου της LCD. Το Control Unit ενεργοποιεί τη γραφή όταν απαιτείται ώστε η τιμή του C να εμφανιστεί στην οθόνη.

Η ALU λαμβάνει τις δύο εισόδους της (**Input A** και **Input B**) από έναν κοινό πολυπλέκτη. Ο πολυπλέκτης αυτός διαθέτει τις ακόλουθες πηγές:

- ACC1
- ACC2
- A
- B

Το **Control Unit** επιλέγει ποια από αυτές τις τέσσερις τιμές θα σταλεί στην είσοδο A και ποια στην είσοδο B της ALU, επιτρέποντας ευέλικτο σχηματισμό πράξεων.

7 Control Unit

Το Control Unit είναι το τμήμα του επεξεργαστή που διαβάζει την εντολή από το IR και καθορίζει όλα τα απαραίτητα σήματα για την εκτέλεσή της. Η λειτουργία του οργανώνεται σε διαδοχικές καταστάσεις (FETCH, DECODE, EXECUTE) που αλλάζουν σε κάθε θετική ακμή του ρολογιού.

Βασικές λειτουργίες

1. **FETCH**
    a. Φορτώνεται η επόμενη εντολή από τη ROM.
    b. Αυξάνεται ο PC.
    c. Το IR ενεργοποιεί το `ir_ld`.
2. **DECODE**
    a. Εξετάζονται τα 8 MSB της εντολής (opcode).
    b. Το Control Unit επιλέγει ποια σήματα φόρτωσης (A, B, C, ACC1, ACC2) θα ενεργοποιηθούν.
    c. Επιλέγονται τα κατάλληλα MUX:
        i. `data_bus_sel`
        ii. `alu_mux_selA/B`
        iii. `mux_sel` (για το αποτέλεσμα)
        iv. `pc_mux_sel` (για jump)
3. **EXECUTION**
    Για κάθε ομάδα εντολών ενεργοποιούνται διαφορετικές διαδρομές:

a) MOV / MVR

a. Φόρτωση άμεσης τιμής ή τιμής register στο επιλεγμένο register.
b. Ενεργοποίηση μόνο του αντίστοιχου X_ld σήματος.

b) LD / ST

c. Η LD επιλέγει RAM, βάζει τη διεύθυνση, περιμένει τα δεδομένα και τα γράφει σε register.
d. Η ST παίρνει τιμή από register και γράφει στη RAM στη διεύθυνση που δίνει η εντολή.

c) ALU εντολές τύπου 1 (ADD1, SUB1, ...)

e. Πηγή: ACC1 και ACC2.
f. Επιλογή πράξης με `alu_sel`.
g. Αποθήκευση αποτελέσματος στο register που δείχνει η εντολή.
h. Ενημέρωση flags.

d) ALU εντολές τύπου 2 (ADD2, SUB2, ...)

i. Πηγές: Επιλογή από A, B, ACC1, ACC2 μέσω `alu_mux_selA/B`.
j. Ίδια λογική με τις τύπου 1.

e) SHIFT (SHR / SHL)

k. Επιλογή καταχωρητή, πλήθος shifts και αποθήκευση του αποτελέσματος πίσω στο ίδιο register.
l. Δεν γίνεται ενημέρωση flags.

f) CMP και όλες οι παραλλαγές CMPRR / CMPRROM / ...

m. Πραγματοποιείται αφαίρεση για να παραχθούν Carry/Zero.
n. Δεν αποθηκεύεται αποτέλεσμα, μόνο flags.

g) Jumps

o. Το Control Unit ελέγχει τα flags.
p. Αν η συνθήκη είναι σωστή, γράφει στο PC τη νέα διεύθυνση μέσω `pc_mux_sel`.

h) IN / PRT / CLR / HLT

q. **IN**: Φόρτωση χρήστη (user_input) σε ACC1.
r. **PRT**: Ενεργοποίηση C register → LCD.
s. **CLR**: Μηδενισμός όλων των registers.
t. **HLT**: Σταμάτημα εκτέλεσης (μένει σε κατάσταση HALT).

Σήματα που ελέγχει

Το Control Unit χειρίζεται:

- **Load enable** για όλους τους registers (A, B, C, ACC1, ACC2, IR)
- **RAM διεύθυνση & write enable**
- **ROM επιλογή**
- **ALU selector**
- **Multiplexers για data path**
- **PC increment / PC write**
- **Flag load**
- **LCD trigger**

Ροή λειτουργίας

1. **FETCH → FETCH2 → DECODE**
2. Επιλογή κατάλληλης κατάστασης ανάλογα με το opcode (MOV_1, LD_1, alu_1, shift, jumps...)
3. Εκτέλεση μικροεντολών
4. Επιστροφή στο FETCH1 για την επόμενη εντολή

8 Assembler

Ο επεξεργαστής χρησιμοποιεί ROM εντολών 32-bit, επομένως απαιτείται ένας Assembler που μεταφράζει ένα απλό Assembly σε machine-code και στη συνέχεια σε VHDL μορφή κατάλληλη για φόρτωση στη ROM.
Ο Assembler υλοποιήθηκε σε Python και αποτελείται από τέσσερα βασικά υποσυστήματα:

8 Assembler

Ο Assembler λειτουργεί σε δύο στάδια (two-pass assembler):

Pass 1 – Ανάλυση & Συλλογή Συμβόλων

- Εντοπίζει labels.
- Αποθηκεύει τις διευθύνσεις τους σε έναν πίνακα συμβόλων (symbol table).
- Ελέγχει τη σύνταξη κάθε εντολής.

Pass 2 – Μετάφραση σε Machine Code

- Μεταφράζει κάθε εντολή στο αντίστοιχο opcode.
- Μετατρέπει καταχωρητές, τιμές και διευθύνσεις σε 8-bit fields.
- Αντικαθιστά labels με τις τελικές διευθύνσεις.
- Δημιουργεί το τελικό 32-bit instruction word:

Parsing & Tokenization

Ο Assembler αναλύει κάθε γραμμή Assembly και την μετατρέπει σε tokens:

Παράδειγμα:

```
ADD  A, B
Tokenization -> ["ADD", "A", "B"]
```

Υποστηρίζει επίσης:

- Σχόλια (`; text`)
- Labels (π.χ. `loop:`)
- Σταθερές σε δεκαδικό bin ή hex (`20`, b00100100, x14,)

Οι κενές γραμμές αγνοούνται.

Error Reporting

Ο Assembler εντοπίζει:

- άγνωστη εντολή
- λάθος αριθμό operands
- μη έγκυρο όνομα register
- overflow immediate (> 255)
- label που δεν έχει οριστεί

Αναφέρει το σφάλμα με αριθμό γραμμής, π.χ.:

```
Error on line 14: Unknown register 'G'
```

9 Παραδείγματα κώδικα

Στην ενότητα αυτή παρουσιάζεται ένα πλήρες παράδειγμα προγράμματος σε Assembly που υλοποιήθηκε για τον επεξεργαστή. Το πρόγραμμα εκτελεί σύνθετες λειτουργίες όπως διαχείριση μνήμης, αριθμητικές πράξεις, εντολές άλματος (jumps) και χρήση της οθόνης για την εκτύπωση του ονόματος "GIORGO PAPAMICHAIL".

Συγκεκριμένα, ο κώδικας:

- Αποθηκεύει και ανακτά δεδομένα από τη μνήμη RAM (χρήση μεταβλητών *d1, *d2).
- Χρησιμοποιεί εντολές ολίσθησης (Shift) και λογικές πράξεις (AND, XOR) για τη διαμόρφωση χαρακτήρων.
- Υλοποιεί βρόχους (loops) για την επαναληπτική εκτύπωση χαρακτήρων (π.χ. "PAPA").
- Διαχειρίζεται τα Flags (Carry, Overflow) για τη λήψη αποφάσεων ροής.

Κώδικας Προγράμματος:

```
*d1 : b00001111

*d2 : xAf

add A

;G I O R G O   P A P A M I C H A I L

;47 49 4F 52 47 4F 20 50 41 50 41 4D 49 43 48 41 49 4C

;Store 'G' to d1

MOV A, x47

mov *d1, A

;store 'I' - 5 to d2

mov ACC1, b01001001

mov acc2, 5

sub A

mov *d2, A

;print G

mov C, *d1
```

```
PRT

;if jump fails we print dump

mov C, x47

jmp save_the_print

PRT

;So we thake the d2 and see if its lowere than 'I' if it is then we add to it 5 and print it else terminate program

- save_the_print

;cmp *d2, b01001001

;jb print_I

;PRT

;jmp terminate

- print_I

mov ACC1, *d2

mov ACC2, 5

add B

mov c, b

prt

; shit the O Left and right then print

mov ACC1, x4f

shl ACC1, 1 ;10011110

shr acc1, 1 ;01001111

mov acc2, 0

add C
```

```
prt
;NOW we have GIO
;if we have an overflow print 'R'
mov A, 255
mov B, 255
add A, B; now A is 11111110
jc overflowmet
mov c, x49
prt
mov c, x30
prt
- overflowmet
;A = 11111110
;'R'=01010010
;        11111110
mov acc1, xAC
xor A, acc1
mov C, A
prt
; print G that was stored at d1
mov C, *d1
mov C, *d1
prt
;print 'O' by doing and to ff and 'o'
```

```
mov acc1, b11111111

mov acc2, b01001111

and c

prt

;printing space wich is x20

mov A, x02

shl A, 4

mov C, a

mov C, a

 Prt

;now we will try to print papa with like a loop

 mov A, 0

jmp printP

 mov C, x30

prt

mov C, x30

prt

mov C, x30

prt

mov C, x30

prt

mov C, x30

prt

- printP
```

```
  mov C, x50
  prt
  jmp printA
- printA
  mov C, x41
  prt
  inc a, a
  cmp A, 1
  jbe printP
;print M
mov C, x4d
prt
;print I
mov C, x49
prt
;print C
mov C, x43
prt
;print H
mov C, x48
prt
;print A
mov C, x41
prt
```

```
;print I

mov C, x49

prt

;now the user must succed his name

in c

mov c,c

prt

-terminate

HLT
```